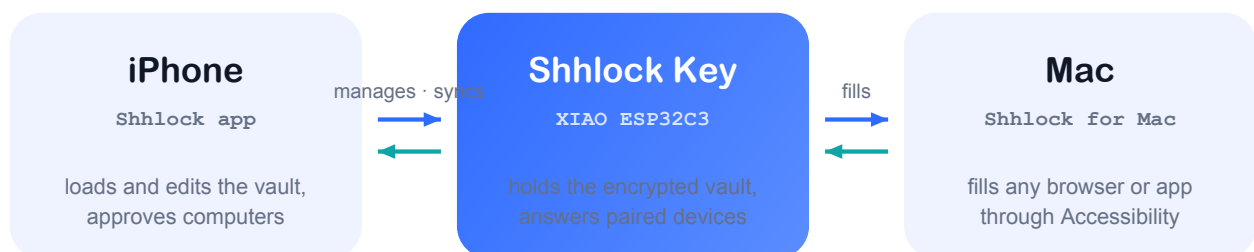




Shhlock

Your passwords, on a key you carry. Any browser. No phone in hand. No cloud.

Shhlock is a small Bluetooth key (a Seeed Studio XIAO ESP32C3) that holds your logins, an iPhone app that manages them, and a menu-bar app for the Mac that fills them into any browser or app while the key is near. Walk away with the key and the computer has nothing.



every message sealed end-to-end · P-256 + AES-256-GCM · the key opens only for devices you paired

1 Set up once

Pair the phone by pressing the key's button. Import your Chrome or Safari passwords. Approve your Mac with a six-digit code.

2 Then just sign in

Open a login page. One matching login fills itself; several show a short list under the field to click.

3 Walk away

Bluetooth reaches a few metres. Take the key, and the Mac can fill nothing — it never stored a password.

This guide: 2 · How it works 3 · Why it is safe 4 · Set it up 5 · Test it, ship it, fix it

How it works

The phone is needed once, to set things up. Afterwards the key and the Mac work on their own.

When you open a login page on the Mac

- 1 **Shhlock for Mac notices the field.** Through macOS Accessibility it sees that a password or e-mail field has focus, in any app.
- 2 **It reads the page's address.** From the browser's web area: `https://github.com`. A page cannot lie about it.
- 3 **It asks the key.** A sealed request over Bluetooth: "logins for github.com?" The key matches by domain and answers, sealed.
- 4 **It fills — or asks you.** One match fills at once. Several show a small list under the field; click one. Cmd-Shift-F fills on demand.
- 5 **Nothing stays behind.** The Mac keeps only its pairing. Passwords exist in its memory for the moment of filling.



Why not Bluetooth alone?

A Bluetooth device can only type into a computer, like a keyboard. It cannot see which site is open or draw a list to pick from. Something on the computer must do that — and a menu-bar app that works in every browser is lighter than an extension per browser.

What the phone does

Loads the vault

Import your Chrome or Safari export, add logins by hand, edit and delete. Every change syncs to the key while it is in reach.

Approves computers

A new Mac shows a six-digit code; the phone shows the same one. Tap "Same code" and the Mac may use the key from then on.

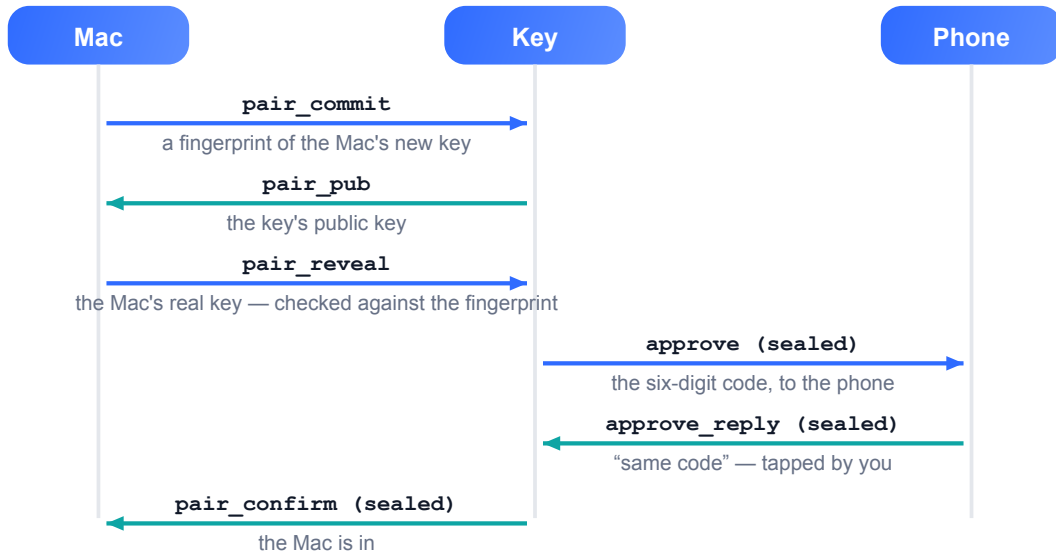
Keeps a copy

The phone holds its own encrypted copy. Lose the key: reset a new one, pair, and the vault is back in a minute.

Why it is safe

The vault sits on a device you could drop in the street. The design assumes that will happen.

Approving a computer: a code that cannot be faked



The Mac commits to its key before it learns the key's, so nobody in between can grind for a matching code. The phone pairs differently: it must press the physical button on the key.

Who holds what

PART	WHAT IT HOLDS	WHAT IT NEVER HAS
● Shhlock Key	The vault, AES-256-GCM. The vault key exists only in RAM after a paired device unlocks it.	The vault key in the clear. Stolen alone, it is unreadable.
● Mac	Its pairing: a session key and a 32-byte unlock secret, in the login keychain.	No vault. Passwords only in memory while filling.
● iPhone	Its own encrypted copy, Face ID locked, key in the Keychain (this device only).	Never in iCloud, never in a backup.
● Chrome extension	Optional. Same pairing scheme as the Mac app.	Nothing on disk beyond its pairing.

Also built in

- A rising counter in every message: recorded traffic cannot be replayed.
- Logins are matched by registrable domain; `evil.github.io` never sees `you.github.io`.
- 40 requests a minute per device.
- Hold the key's button 8 s: factory reset.

Honest limits of version 2

- No forward secrecy between a device and the key.
- The ESP32's flash is not encrypted: an attacker with the key and a paired Mac's keychain could read the vault.
- No cloud copy by design — use Settings → Export a backup.

Set it up

1 Flash and power the key

Plug the XIAO ESP32C3 into the Mac once to flash it. Afterwards any USB power will do — a battery in your bag.

```
firmware/flash.sh
```

2 Pair your phone with the key

Install Shhlock from TestFlight. **Key** → **Pair this phone with the key**, then press the small button on the XIAO board when the app asks. That press is what proves the key is yours.

3 Load your passwords

Chrome: `chrome://password-manager/settings` → **Export passwords**. Safari: File → Export → Passwords. AirDrop the .csv to the phone, then **Settings** → **Import from Chrome or Safari**. The vault syncs to the key on its own. Delete the .csv afterwards — it is plain text.

4 Install Shhlock for Mac

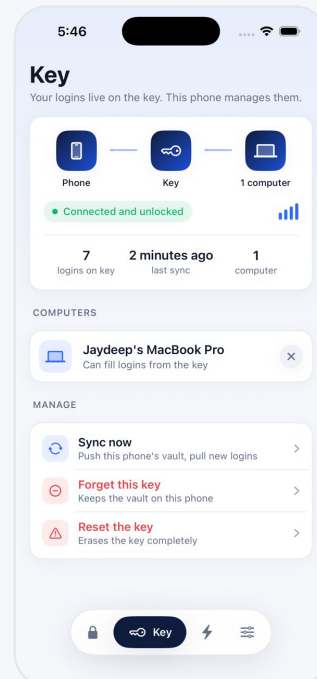
```
scripts/install-mac-app.sh
```

Allow Bluetooth when asked. From the padlock in the menu bar choose **Set up Shhlock...** and allow Accessibility (System Settings opens; switch Shhlock on).

5 Approve the Mac

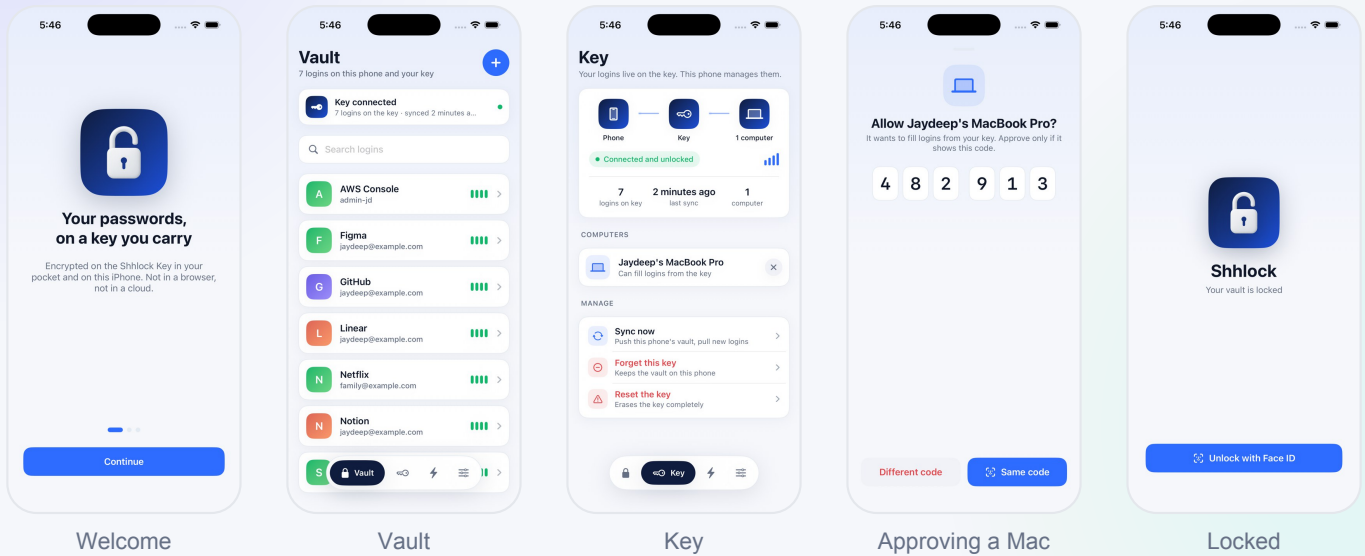
With the phone app open, click **Pair with my key** on the Mac. Both show six digits. Tap **Same code** on the phone, click **Same code** on the Mac. Done — the phone can go back in your pocket.

Nothing is deleted from Chrome. Turn off Chrome's own "offer to save passwords" so only Shhlock asks.



The Key tab after pairing and syncing.

Test it, ship it, fix it



Welcome

Vault

Key

Approving a Mac

Locked

Try it

- 1 First fill.** Open a site you imported. The fields fill and the padlock in the menu bar shows “Ready · N logins on your key”.
- 2 Several logins.** On a site with two accounts a list appears under the field. Click one.
- 3 Walk away.** Take the key out of the room. The menu says “Looking for your Shhlock Key”; nothing fills. Come back: ready again within seconds.
- 4 Without the phone.** Switch the phone off. Filling on the Mac keeps working — the vault is on the key.
- 5 Remove a computer.** On the phone, Key → ☐ next to the Mac. The Mac gets no answers until approved again.

Automated checks

```
cd core && swift test
firmware/flash.sh --test
cd core && swift run shhlock-keytest
```

Unit tests · then the real key over USB: pairing, approval, a 123-login sync, matching, save, replay, reboot — 21 checks. Flash without `--test` afterwards.

TestFlight

Sign in under Xcode → Settings → Accounts; the app record in App Store Connect uses bundle ID `com.codecrackjd.safely`. Then:

```
scripts/testflight.sh --upload
```

If something is off

Menu: “Needs Accessibility” — System Settings → Privacy & Security → Accessibility → Shhlock on. Re-do after reinstalling.

Pairing the Mac stalls — The phone app must be open (it approves). Also check the phone shows “Connected and unlocked”.

“Looking for your Shhlock Key” — Is it powered? System Settings → Privacy → Bluetooth → Shhlock on.

Nothing fills on a site — Is the login under that domain in the vault? Try Cmd-Shift-F. Some pages need the fallback typing, which takes a second.

Mac log: Set up Shhlock... → Activity log · Key log: `arduino-cli monitor -p /dev/cu.usbmodem1101 -c baudrate=115200` · Protocol: docs/PROTOCOL.md